

ECEN 428/722 - Post Lab Report

Lab Number: 2

Lab Title: CORDIC Circuit Design using Vitis HLS

Section Number: 602

Student's Name: Sai Vikhyat Pattar

Student's UIN: 936001854

Date: 9/25/2025

TA: Cristhian Roman Vicharra

Objective:

Execute the complete Vivado FPGA design flow using a provided FIR filter and a self-written IIR filter, then analyze how adding **pin**, **timing**, and **placement** constraints modifies placement and timing. Specifically, implement the IIR RTL, run behavioral simulation, implement both filters under multiple constraint scenarios, capture device views, and summarize observations.

Setup:

1. **Project/Files/Part:** Create an RTL project in Vivado; add fir.v and multiply.v as sources and zybo_constraints.xdc as a constraint file; select part **xc7z020clg400-1** (ZYBO Z7 family). The code bundle includes fir_tb.v and iir_tb.v; complete iir.v.
2. **Clocking:** Use the Zybo's **125 MHz** external clock on **pin K17**. Generate a **200 MHz** internal clock via **Clocking Wizard (PLL)** configured as single-ended input; enable reset and locked ports and instantiate it as clk_wiz_0 in the filter designs.
3. **Simulation:** Add the provided testbench (e.g., fir_tb.v) as a simulation source and run **Behavioral Simulation**.
4. **Constraints you will apply:**
 - **Pin mapping** with set_property ... [get_ports ...] in zybo_constraints.xdc.
 - **Timing** using create_clock (mandatory on the **PLL input** clock; Vivado propagates to PLL outputs).

Design:

1. **FIR (given):** fir.v plus multiply.v implements the Figure-1 FIR architecture; use it to drive the flow and as a template for clocking.

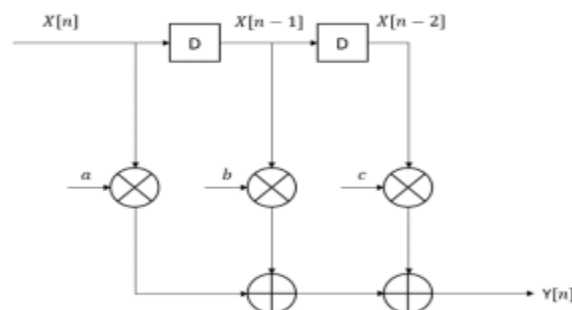


Figure 1: FIR Filter

2. **IIR (you implement):** Write iir.v for the Figure-2 IIR (e.g., p=1, q=2) and **instantiate** clk_wiz_0 the same way as in the FIR design.

$$y[n] = \sum_{k=0}^p b_k x[n - k] + \sum_{k=1}^q a_k y[n - k]$$

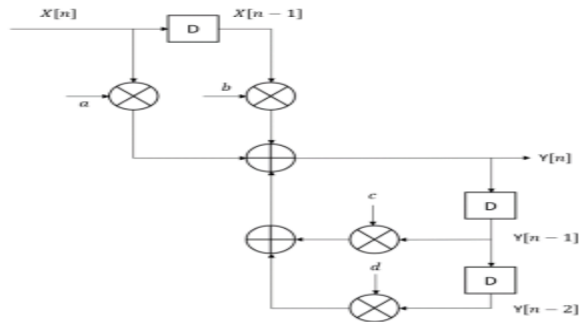


Figure 2: IIR Filter

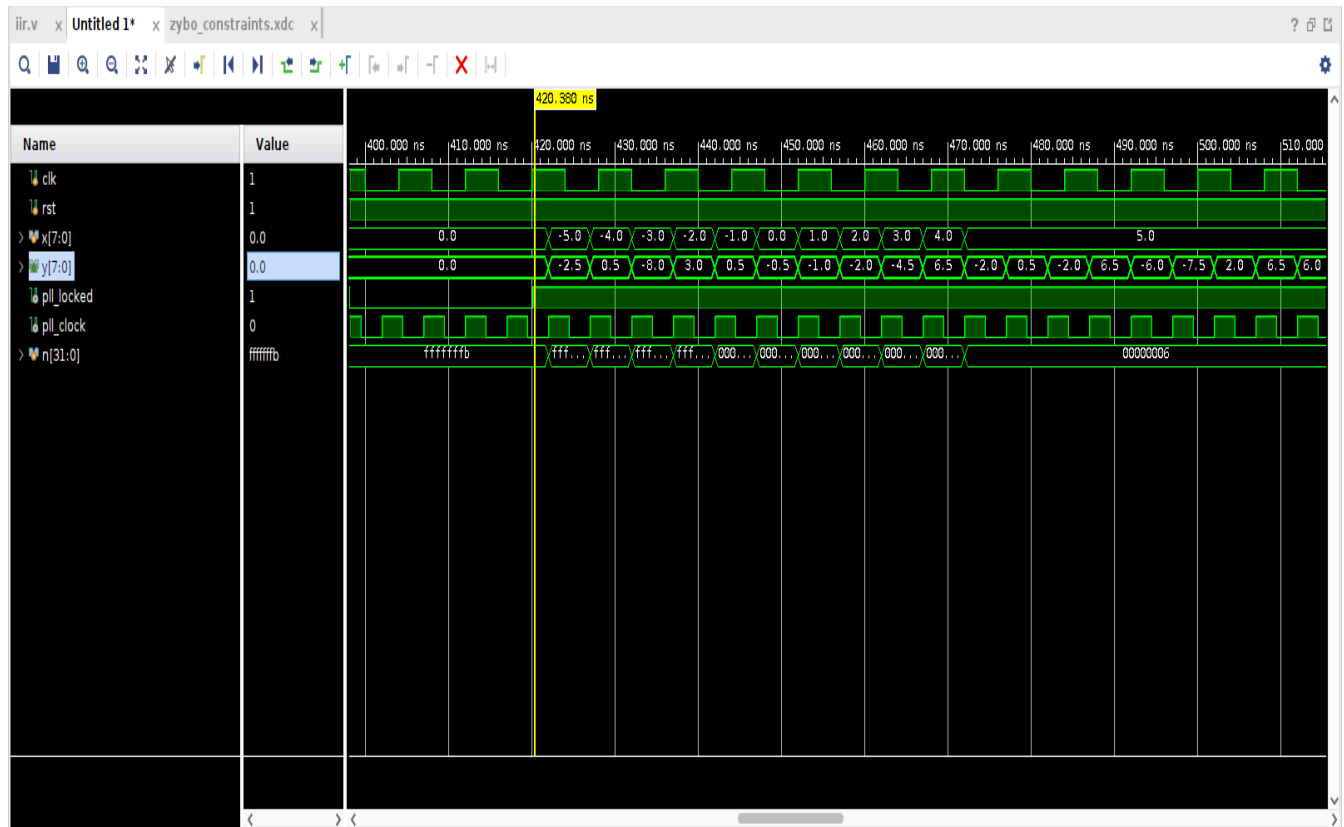
Implementation:

1. **Synthesis → Implementation (Place & Route)** for each scenario below.
2. **Device View:** Inspect placement and routing; take screenshots for the report.
3. **Constraint scenarios to run and capture:**
 - **No user constraints** (baseline).
 - **Pin constraints only** (placement shifts toward I/O).
 - **Pin + timing constraints** (elements move to shorten paths; improves slack).
 - **Pin + timing + placement (pblock)** (logic confined to the user region).
For both FIR and IIR, capture the effect of placement with and without pin/timing constraints and include in the report

Results discussed:

The IIR is verified directly from the timing waveforms. After `rst` is de-asserted and `pll_locked` stays high, the input `x` is swept in Q4.4 from `-5.0` to `+5.0` (see the value bubbles on `x[7:0]`). The output `y[7:0]` changes one clock later—showing the expected **one-sample latency** from the internal delay registers—and its numeric bubbles match the IIR difference equation used in RTL: each `y[n]` equals the fixed-point sum of the current input term(s) and the feedback terms from `y[n-1]` and `y[n-2]` (within normal Q4.4 rounding). For example, around the highlighted cursor (~420 ns) when `x` steps to `-5.0`, the next cycle shows `y ≈ -2.5`, and subsequent samples follow the recursive response sequence visible in the bubbles. No X/Z states appear, the clock is clean, and the consistent agreement between computed values and displayed `y` across the sweep confirms functional correctness.

IIR Filter behavioral simulation waveform:



The further results discussed will consist of the implementation without pipelined registers and with pipelined registers with a comparison.

Results without pipelined registers include:

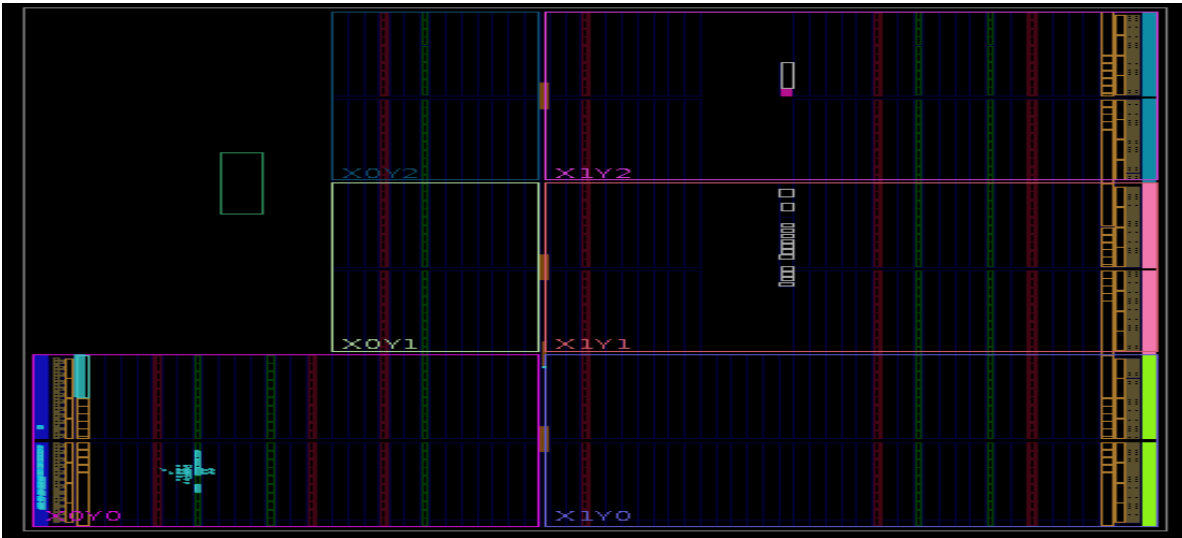
1. IIR Filter without any constraints
2. IIR Filter with pin constraints
3. IIR Filter with pin and timing constraints
4. IIR Filter with pin, timing and placement constraints

IIR Filter without any constraints

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -2.484 ns	Worst Hold Slack (WHS): 0.187 ns	Worst Pulse Width Slack (WPWS): 2.000 ns
Total Negative Slack (TNS): -16.918 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 8	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 16	Total Number of Endpoints: 16	Total Number of Endpoints: 30

Timing constraints are not met.

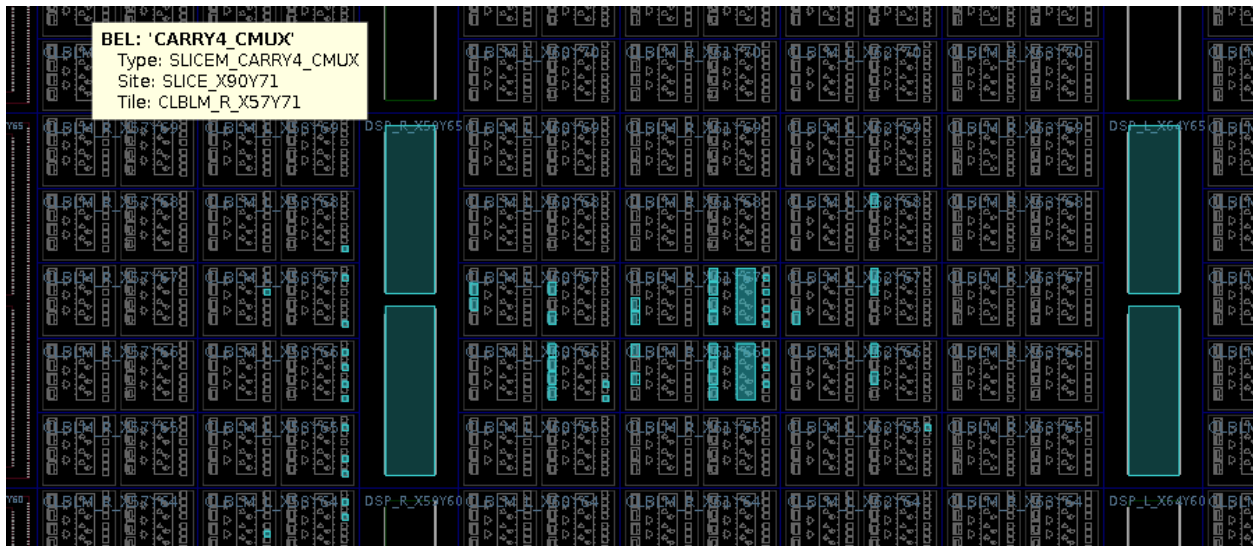
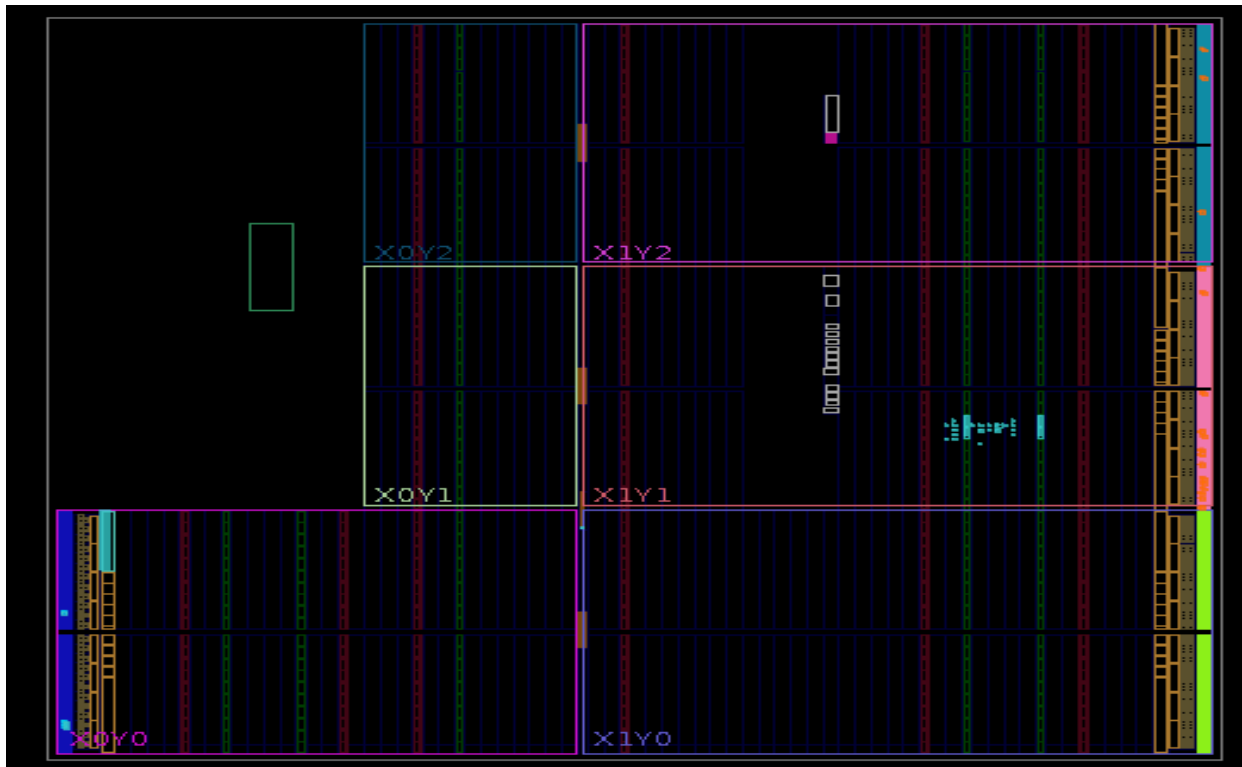


IIR Filter with pin constraints

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -2.482 ns	Worst Hold Slack (WHS): 0.228 ns	Worst Pulse Width Slack (WPWS): 2.000 ns
Total Negative Slack (TNS): -17.038 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 8	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 16	Total Number of Endpoints: 16	Total Number of Endpoints: 30

Timing constraints are not met.

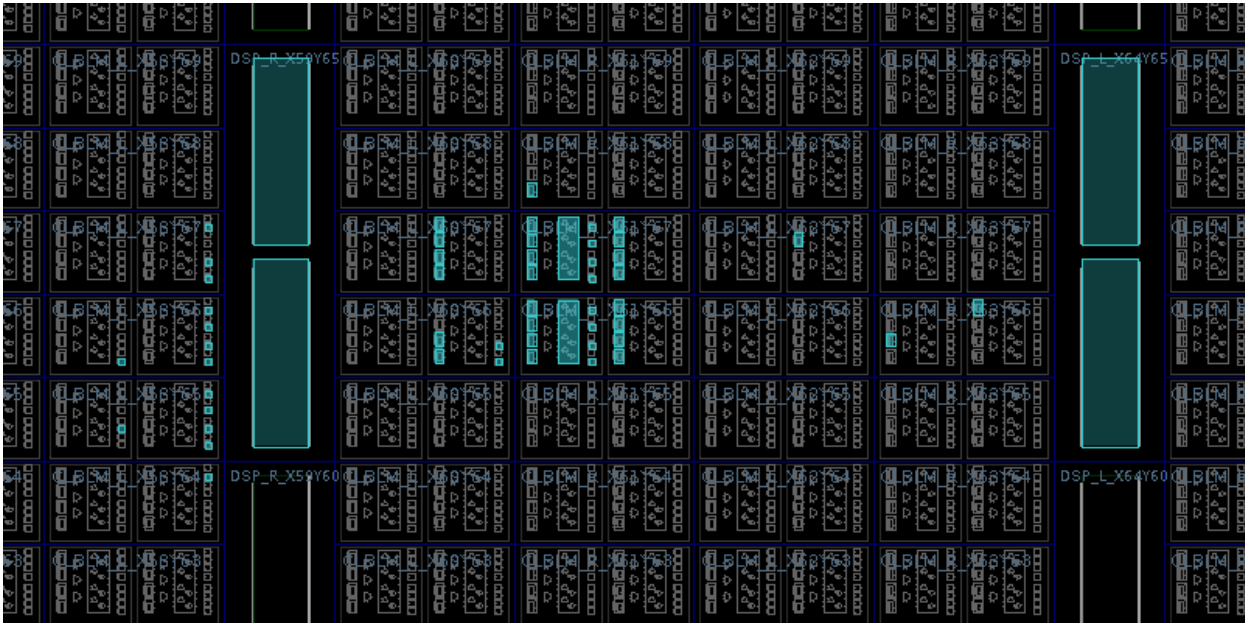
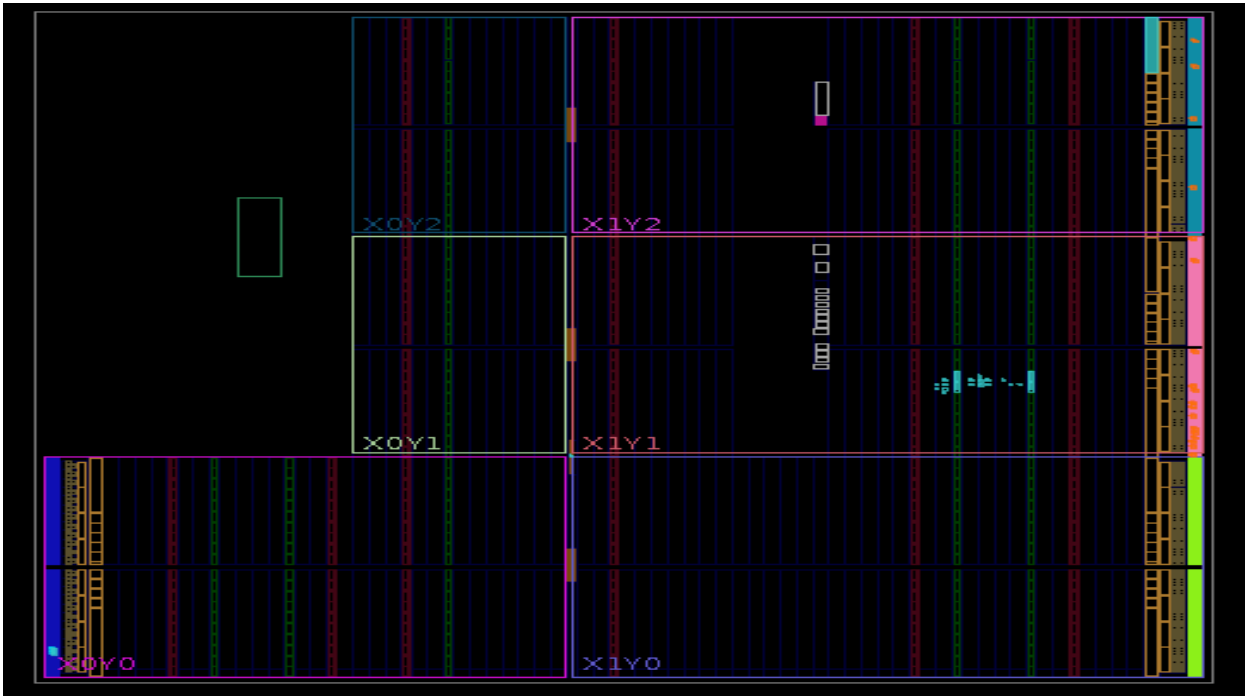


IIR Filter with pin and timing constraints

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -2.383 ns	Worst Hold Slack (WHS): 0.194 ns	Worst Pulse Width Slack (WPWS): 2.000 ns
Total Negative Slack (TNS): -15.893 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 8	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 16	Total Number of Endpoints: 16	Total Number of Endpoints: 30

Timing constraints are not met.

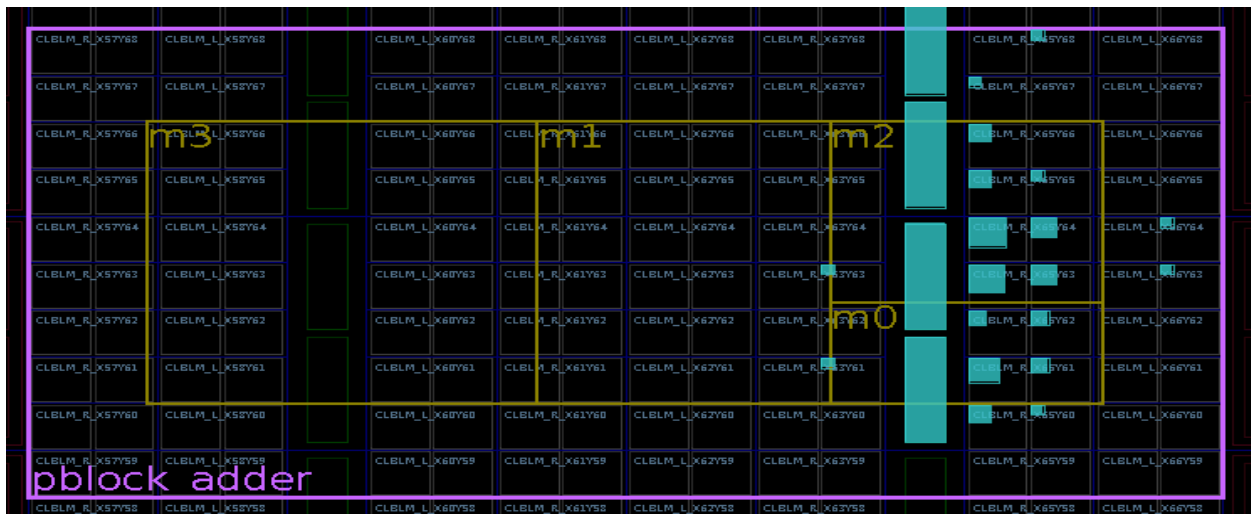
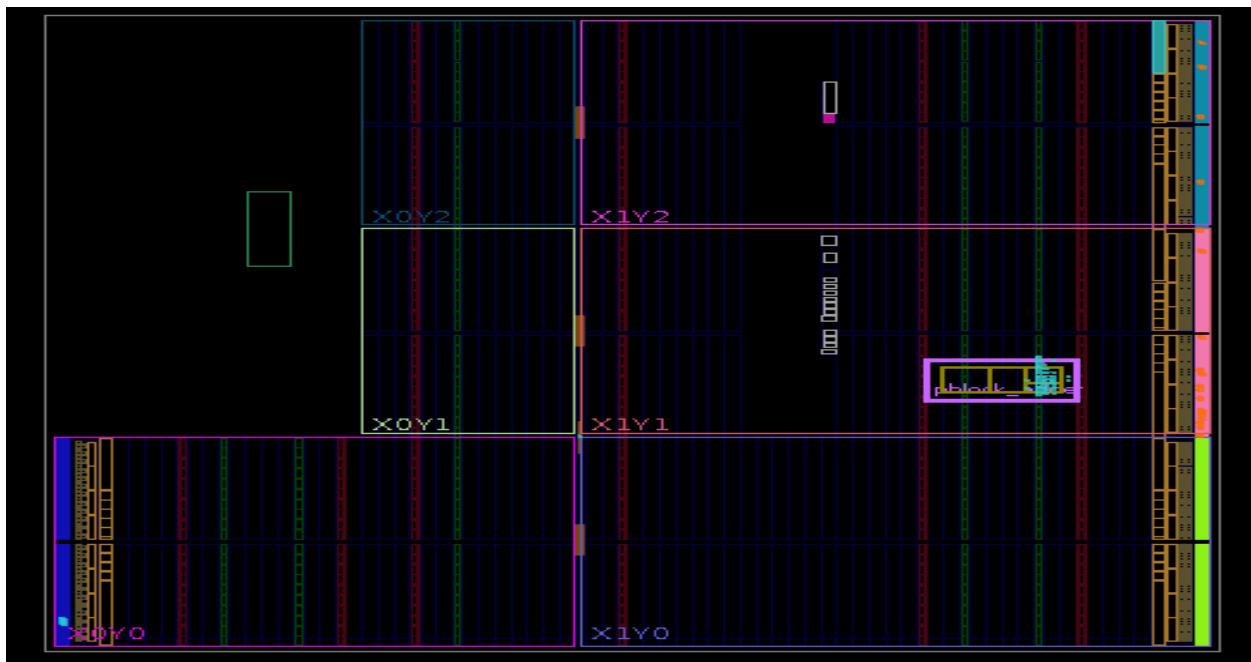


IIR Filter with pin, timing and placement constraints:

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -2.274 ns	Worst Hold Slack (WHS): 0.216 ns	Worst Pulse Width Slack (WPWS): 2.000 ns
Total Negative Slack (TNS): -16.229 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 8	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 16	Total Number of Endpoints: 16	Total Number of Endpoints: 30

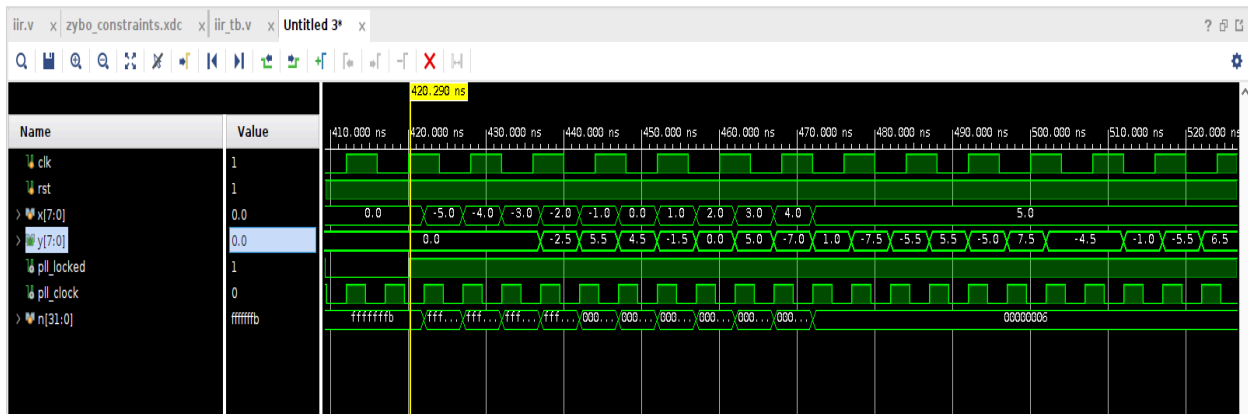
Timing constraints are not met.



Results with pipelined registers include:

1. Behavioral simulation waveform
2. IIR Filter with all constraints

Behavioral simulation:



Behavioral simulation looks “wrong” after pipelining because the extra registers weren’t added uniformly across all paths, so the operands that reach the final adder no longer represent the **same time index**. In this IIR, the multiplier results are registered but the feedback path is also updated in that same cycle, making y_1 lead the official output (y) by a clock. The adder then combines $a*x[n-1]$, $b*x[n-2]$, $c*y[n]$ (effectively), and $d*y[n-1]$ instead of the intended $a*x[n]$, $b*x[n-1]$, $c*y[n-1]$, $d*y[n-2]$. That misalignment changes the recursion (hence the waveform), so even though timing meets constraints, the simulated output diverges from the non-pipelined reference. The fix is to balance the pipeline on **every** path and take feedback after the **same** pipeline boundary as the reported output.

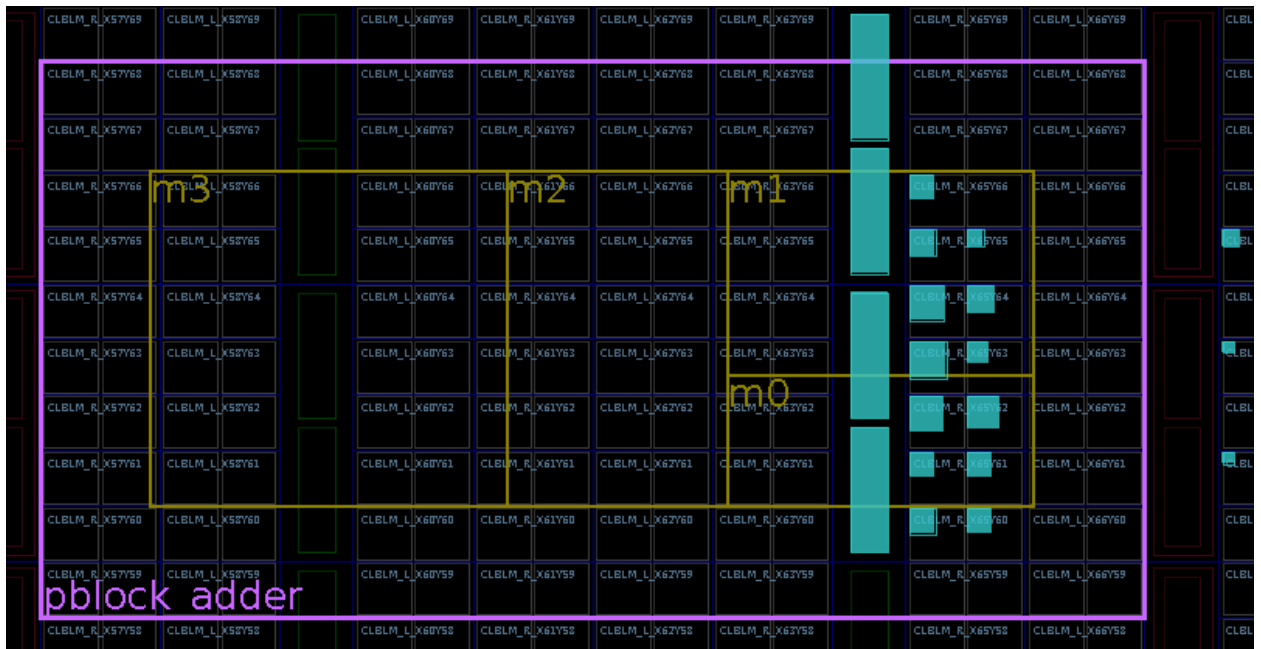
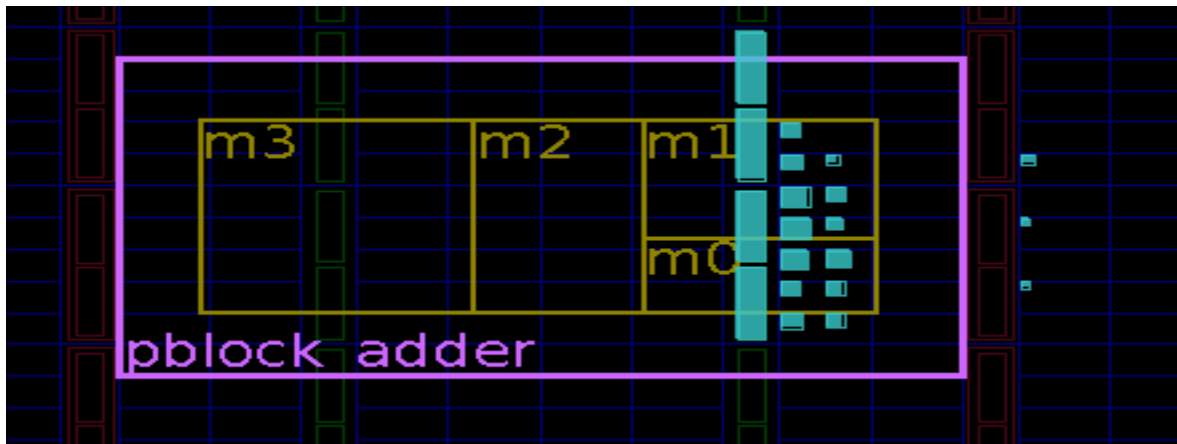
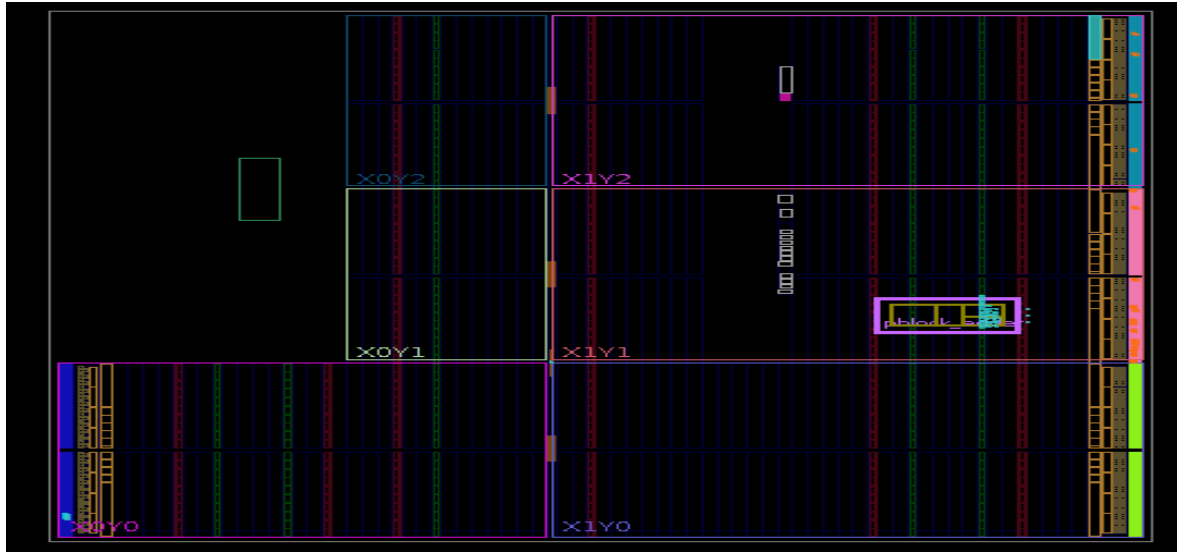
IIR Filter with all constraints

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.281 ns	Worst Hold Slack (WHS): 0.172 ns	Worst Pulse Width Slack (WPWS): 2.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 68	Total Number of Endpoints: 68	Total Number of Endpoints: 51

All user specified timing constraints are met.

The design reports **TNS = 0 ns** because **no path violates setup timing**. Total Negative Slack is the sum of all negative slacks across endpoints; when every endpoint has **non-negative slack**, the sum is zero. Here the **WNS = +0.281 ns** (the critical setup path still has 0.281 ns margin), and the report shows **0 failing endpoints (68 checked)**. Hold timing is also clean (**WHS = +0.172 ns, THS = 0 ns**), and there are no pulse-width violations. In short, proper pipelining and correct clock constraints mean every register-to-register path meets the required period, so TNS is zero and “All user specified timing constraints are met.”



Inference:

Non-pipelined IIR (functional but fails timing)

- Behavioral simulation matches the intended recursion
 $y[n] = a*x[n] + b*x[n-1] + c*y[n-1] + d*y[n-2] \rightarrow$ waveforms align with the testbench; functionality is intact.
- The datapath (four multiplies + sum) sits in one cycle, so the critical path is too long for the target clock $\rightarrow$ **setup timing violations** (negative slack; non-zero TNS) even though the outputs are correct.

Pipelined IIR (meets timing but breaks functionality)

- Inserting registers after the multipliers shortens the critical path $\rightarrow$ timing closes (e.g., **WNS > 0, TNS = 0** in the report).
- But the pipeline is **unbalanced**: feedback and feed-forward paths are not delayed equally, so the adder combines **misaligned time indices** (effectively using $c*y[n]$ and shifted x terms).
The design now behaves like
 $y[n] = c*y[n] + d*y[n-1] + a*x[n-1] + b*x[n-2]$
instead of the target equation $\rightarrow$ behavioral waveforms no longer match, i.e., **functionality is wrong**.
- **Fix (brief):** balance registers on **all** paths and take feedback at the **same pipeline boundary** as the reported output (or use a transposed/DF-II structure that pipelines cleanly).

Conclusion:

This lab walked through the full Vivado flow—RTL, simulation, synthesis, implementation, timing, and bitstream—using a provided FIR and a self-written IIR. We verified that the **non-pipelined IIR** correctly implements the target recursion in behavioral sim, but its long combinational path failed setup timing at the target clock. Introducing **pipeline registers** resolved timing (WNS > 0, TNS = 0), yet the first attempt misaligned feed-forward and feedback paths, altering the intended difference equation and producing incorrect waveforms. This contrast underscored two key lessons: (1) **constraints** (pin, timing, placement) materially shape placement and slack, and (2) **pipelining must be balanced**—all operands entering a stage must reference the same sample index, and feedback must be tapped at the same pipeline boundary as the reported output. Overall, the exercise reinforced best practices for clocking, constraint management, and function-preserving pipelining on FPGA designs.